T.C.

FATİH SULTAN MEHMET VAKIF ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM22374 – Bilgisayar Ağları

2 Kişilik Satranç Oyunu

Hazırlayan:	Muhammed Berat AYDEMİR (GitHub: berat0642) & Muhammed Tahir YANMAZ (GitHub: MTYZ0)
Öğrenci No:	2021221372 & 2121221372
Öğretim Elemanları:	Ertuğrul İslamoğlu

İSTANBUL
2026

1. Proje Özeti

Bu proje kapsamında, iki farklı istemcinin uzak bir sunucu üzerinden karşılıklı olarak oynayabildiği, gerçek zamanlı bir çevrimiçi satranç oyunu geliştirilmiştir. Proje gereksinimlerine uygun olarak uygulama dili olarak Python kullanılmış, kullanıcı arayüzü (GUI) ise PyQt5 (Qt) kütüphanesi ile tasarlanmıştır. İstemciler (oyuncular), Amazon Web Services (AWS) üzerinde barındırılan merkezi bir sunucu ile IP adresi ve port (16.171.69.82:5002) üzerinden TCP/IP protokolü kullanılarak haberleşmektedir.

2. Mimari ve Kullanılan Teknolojiler

Proje, klasik İstemci-Sunucu (Client-Server) mimarisi üzerine inşa edilmiştir:

- **İstemci (Client):** Oyuncuların etkileşime girdiği grafiksel arayüzü barındırır. Python ve PyQt5 ile geliştirilmiş olup, son kullanıcı bakış açısıyla tamamen kullanıcı dostu bir arayüze sahiptir. Oyun kuralları, hamle doğrulamaları ve şah/mat durumları istemci tarafında hesaplanır. Ayrıca, piyonların son sıraya ulaştığında vezire terfi etmesi (piyon terfisi), geçersiz hamle (şah altında bırakma) uyarıları, şah/mat/pat durumlarının tespiti ve yenen siyah/beyaz taşların arayüzde ayrı ayrı gösterimi gibi gelişmiş oyun mekanikleri istemci tarafında yönetilmektedir.
- **Sunucu (Server):** AWS EC2 (Ubuntu) üzerinde koşan, grafik arayüzü bulunmayan bir konsol uygulamasıdır. Gelen oyuncu bağlantılarını dinler, eşleştirir ve aralarındaki hamle/durum mesajlarını (relay) iletir. Birden fazla oyun oturumunu (session) aynı anda, birbirine karışmadan yönetebilecek kapasitededir.
- **Ağ İletişimi:** İletişim, Python'ın yerleşik socket ve threading kütüphaneleriyle TCP üzerinden sağlanır. Ağ dinleme işlemleri, ana GUI'nin donmasını engellemek amacıyla ayrı iş parçacıklarında (thread) yürütülmektedir.

3. Uygulama Modülleri ve Geliştirme Detayları

a. İstemci Modülleri:

i. *client.py* (Kullanıcı Arayüzü): İstemci uygulamasının ana modülüdür. Başlangıç ekranı, oyun tahtası (ChessBoardWidget) ve oyun sonu ekranı arasında geçişleri QStackedWidget ile sağlar. Thread-safe sinyal mekanizması (pyqtSignal) kullanılarak ağ üzerinden gelen mesajlar GUI güncellemelerine dönüştürülür. Program kapatılmadan oyunun tekrar oynanmasına izin verecek şekilde tasarlanmıştır. Başlangıç ekranında oyuncular bağlanılacak sunucu IP adresi ve port numarasını dinamik olarak girebilirler; girilmemesi durumunda config.ini dosyasındaki varsayılan değerler kullanılır. Ayrıca oyuncu eşleştirmesi sırasında 10 saniye boyunca rakip bulunamazsa zaman aşımı (timeout) uygulanarak lobiye dönülür.

ii. *pieces.py* (Oyun Mantığı): Satranç taşları Nesne Yönelimli Programlama (OOP) ilkelerine göre tasarlanmıştır. Piece soyut (abstract) sınıfından türetilen taşlar (Piyon, Kale, At vb.), kendilerine has can_move fonksiyonlarıyla hamle doğrulama işlemlerini gerçekleştirir.

iii. *network_client.py* (Ağ Yöneticisi): İstemcinin sunucuya bağlanması, mesaj okuma döngüsü (_receive_loop) ve sunucuya hamle gönderim işlemlerini üstlenir.

b. *server.py*: Konsol tabanlı olarak çalışan TCP soket sunucusudur. Birden çok istemciye eş zamanlı hizmet sunabilir. Bağlanan oyuncuları GameSession nesneleri altında gruplar. Bir oturum dolduğunda (iki kişi bağlandığında) oyunu başlatır ve taraflara MATCHED:white / MATCHED:black mesajları göndererek renk ataması yapar. Bir oyuncu ayrıldığında diğer oyuncuya OPPONENT_LEFT mesajı ileterek onu yeni bir rakip için bekleme havuzuna alır.

4. AWS Dağıtımı (Deployment)

Sunucu uygulamasının bulut ortamında kesintisiz çalışabilmesi için deploy-to-aws.sh isminde bir Bash betiği hazırlanmıştır. Bu betik:

1. SSH üzerinden AWS EC2 instance'ına bağlanıp gerekli Python 3 güncellemelerini yapar.
2. server.py dosyasını sunucuya kopyalar.
3. Sunucu uygulamasını arka planda sürekli çalışır tutmak ve olası çökmelerde otomatik yeniden başlatmak için bir systemd servisi (chess-server.service) kurar ve aktifleştirir.

5. İletişim Protokolü ve Mesajlaşma

İstemci ve sunucu arasında yeni satır (\n) karakteri ile sonlandırılmış basit metin tabanlı (UTF-8) bir iletişim protokolü kullanılmıştır:

- WAITING_FOR_OPPONENT: Sunucunun, oyuncunun eşleşme beklediğini bildirmesi.
- MATCHED:<color>: Sunucunun oyunun başladığını ve rengi bildirmesi.
- sr,sc->er,ec: İstemcinin yaptığı hamleyi (kaynak satır/sütun -> hedef satır/sütun) ifade eden mesaj yapısı.

6. Proje Kurulumu ve Çalıştırılması

Gönderilen dosyalar değerlendirici bilgisayarında herhangi bir yol (path) veya platform sorunu yaratmadan çalışacak şekilde tasarlanmıştır.

1. Sunucu yapılandırması config.ini dosyasından otomatik okunmaktadır (Mevcut IP: 16.171.69.82).
2. İstemciyi başlatmak için bağımlılıkların (PyQt5) kurulu olduğu bir ortamda python client.py komutu çalıştırılmalıdır.

7. Sonuç

Bu projede Soket programlama, Multithreading, OOP ve modern GUI tasarımı konuları bir araya getirilerek kapsamlı ve eksiksiz bir çevrimiçi oyun sistemi geliştirilmiştir. Proje isterlerinde belirtilen AWS barındırma, GUI kısıtlamaları ve çoklu istemci destekleme koşullarının tamamı başarıyla sağlanmıştır.